

Staging deployment runbook

Everything needed to put the API, the database, the cache and the two websites on AWS — and what to do with them afterwards.

Region ap-south-1 (Mumbai) · **Shape** EC2 + Docker Compose behind Caddy
Written 2 October 2026 · **Source of record** docs/04-infrastructure.md, deploy/

Contents

1. What you are building, in plain words	4
2. The concepts, each in a paragraph	5
3. Before you start — accounts, tools, decisions	7
4. AWS groundwork	9
5. RDS — the database	12
6. Box B — Valkey	15
7. Box A — the API box	19
8. DNS at GoDaddy	22
9. Bring the stack up	24
10. Seed the first super admin	25
11. The websites — S3 and CloudFront	26
12. Verify, end to end	28
13. Caddy — what it is, and every line of the Caddyfile	30
14. Grafana Cloud — logs and metrics	32
15. Load testing	34
16. Scaling	37
17. PostgreSQL — backups and restore	39
18. Valkey — connecting and looking inside	41
19. Production — what is different	43
20. The SMS vendor	45
21. Email	46
22. When something is wrong	47
23. After today	48

Staging deployment runbook

IACE platform — what to do today, in order, with nothing assumed

This is written for somebody who has not deployed to AWS before. Every step says what to click or type, and then why that step exists — because the "why" is what lets you fix it when the step does not work, and something always does not work.

Read §1 and §2 first. They take twenty minutes and they are the difference between following instructions and knowing what you are doing. Then work down §4 to §12 in order, which is the actual deployment. Everything after §13 is the parts you asked to have in one place: Caddy, Grafana, load testing, scaling, backups, production, and the SMS and email vendors.

Where the authority lives. This document is a runbook; it is not the source of record. The sizing and the money are `docs/04-infrastructure.md`. The data model is `prisma/schema.prisma`. The operational short form is `deploy/README.md`. Where this document and one of those disagree, they win and this one is stale.

1. What you are building, in plain words

Three computers and a managed database. That is the whole thing.

	What it is	What runs on it
Box A	One EC2 virtual machine	Caddy, and the API in three containers, and a telemetry shipper
Box B	A second, smaller EC2	Valkey — the fast in-memory store that holds live exam sittings
RDS	A managed PostgreSQL that AWS runs for you	Every durable row: students, tests, questions, marks
S3 + CloudFront	Object storage with a CDN in front	The two websites, and question images

A student's browser does two different things at once. It downloads the **website** — HTML, JavaScript, CSS — from CloudFront, which is a cache spread across the world. And it makes **API calls** — sign in, start test, save answers, submit — to Box A. Those are two separate addresses and two separate systems, and almost every deployment confusion comes from mixing them up.

HOW A REQUEST REACHES THE RIGHT PLACE

Student's browser

DNS lookup at GoDaddy

Two destinations

website files → CloudFront → S3

API calls → Box A → Caddy

Inside Box A

Caddy routes by path

exam container → the live sitting

core container → everything else

worker container → scoring, no HTTP

Data

Valkey on Box B → answers in flight

RDS PostgreSQL → marks, for ever

Why three containers from one image. Node.js runs your code on a single CPU core. One container would therefore use one core of a two-core box and leave the other idle for ever. So the same image is started three times with a different `API_ROLE` each time, and each answers a different set of routes. `exam` takes the live sitting — starting a paper, saving answers, submitting. `core` takes everything else — sign-in, results, the whole admin app. `worker` answers no web requests at all; it sits on a queue and scores papers after they are submitted.

Why the worker is exactly one, always. It runs two repeating jobs — a flush every 60 seconds and a sweep every 120 — and the only thing stopping one tick overlapping the next is a setting that works inside a single process. Two worker containers means two sweeps over the same keys with nothing coordinating them. `exam` and `core` can be scaled to as many copies as you like. The worker cannot.

2. The concepts, each in a paragraph

You do not need to know these deeply. You need to know them well enough that an error message means something.

Region and availability zone. AWS runs data centres in groups. `ap-south-1` is Mumbai; inside it, `ap-south-1a` and `ap-south-1b` are separate buildings. Everything you build today goes in one zone, because traffic between zones costs money and buys availability you are not using yet. You will still create a subnet in a second zone, for one reason only: RDS refuses to be created without two.

VPC and subnets. A VPC is your own private network inside AWS, with its own address range — `10.0.0.0/16`, which means every address from `10.0.0.0` to `10.0.255.255`. A subnet is a slice of it, pinned to one zone. A **public** subnet is one whose route table sends unknown traffic to an Internet Gateway. That is the only difference. A public subnet does not make anything reachable by itself — three things have to line up before a packet from the internet arrives at your box: it needs a public address, a security group rule that allows it, and a network ACL that allows it.

Security groups. A stateful firewall attached to a machine, not to a subnet. "Stateful" means you only write the inbound rule; the reply is allowed automatically. The important trick: a rule's source can be **another security group** rather than an address range. Box B's rule says "allow 6379 from whatever is in the API security group", so when you replace Box A, the new box inherits the permission by joining the group. You never edit Box B.

Elastic IP. A public IPv4 address that belongs to you rather than to an instance, so it survives the instance being destroyed. It is also your failover mechanism: if Box A dies, you launch a replacement and move the address to it, and the DNS never changes.

Instance profile. An IAM role attached to a machine. Software on that machine can ask a special local address for temporary AWS credentials and get them, scoped to that role. This is why there is no AWS key pair in your environment file: the box proves who it is by being itself.

Docker image versus container. An image is a frozen filesystem — your built app plus Node plus its libraries. A container is one running copy of an image. One image, started three times with different settings, gives you the three roles. `docker compose` is a file that describes several containers and their settings so you start them all with one command.

Docker Compose "service" versus "container". In `deploy/compose.yml`, `exam` is a *service*. If you scale it to four, you get four *containers* all answering to the DNS name `exam` inside Docker's private network. That is what makes scaling invisible to Caddy.

Reverse proxy. A program that accepts a request from the internet and forwards it to one of several programs behind it, picking by some rule. Caddy is yours. It also does TLS: it obtains the HTTPS certificate, decrypts the request, and talks plain HTTP to your containers on the private network.

ACME and Let's Encrypt. Let's Encrypt gives out free certificates, automatically. The "HTTP-01 challenge" works like this: Caddy asks for a certificate for `api.staging.examprep.iace.co.in`; Let's Encrypt replies "prove it — put this token at `http://api.staging.examprep.iace.co.in/.well-known/acme-challenge/<id>`"; Caddy serves it; Let's Encrypt fetches it from the public internet and issues the certificate. **This is why the DNS record must exist and port 80 must be open before you start Caddy.** It is also why an AWS-provided hostname cannot be used: `*.amazonaws.com` is on the Public Suffix List and Let's Encrypt refuses it outright.

DNS record types. An **A** record maps a name to an IPv4 address. A **CNAME** maps a name to another name. You cannot put a CNAME at the root of a domain, but every name you use today is a subdomain, so that limit never bites.

TTL. How many seconds the rest of the internet is allowed to remember a DNS answer. Set it low (600) while you are setting up, so a mistake costs ten minutes instead of a day.

S3 and CloudFront. S3 stores files. CloudFront is a cache in front of it with servers in many cities, which also brings a free HTTPS certificate on its own `*.cloudfront.net` name. The websites are just files, so this is all they

need — there is no server rendering anything.

RDS. AWS runs PostgreSQL for you: backups, point-in-time restore, and growing the disk without downtime. The platform could run Postgres in a container, and should not: the one operation nobody should perform for the first time under pressure is the one holding students' marks.

Valkey. A fork of Redis, same protocol. It holds a live sitting's answers while the exam is in progress, the session tokens, the OTP codes, and the job queue. It is in memory, which is why answers are written there every 25 seconds instead of to Postgres on every keystroke.

BullMQ. The job queue library, which stores its jobs in Valkey. When a student submits, the API puts a "score this attempt" job on the queue and answers the student immediately. The worker picks it up a moment later. This is why 8,000 students can submit in the same minute without the database melting.

3. Before you start — accounts, tools, decisions

3.1 Install these on your laptop

The AWS CLI is **not installed** on this machine — check and install it first, because most of today needs it.

```
# AWS CLI v2
curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "/tmp/AWSCLIV2.pkg"
sudo installer -pkg /tmp/AWSCLIV2.pkg -target /
aws --version

# The Session Manager plugin, so `aws ssm start-session` works
brew install --cask session-manager-plugin
session-manager-plugin          # prints a success banner
```

macOS has no "bundle" zip for the plugin, whatever a Linux guide says — the only official artifact is a signed `.pkg`, and the bundle URL answers with an S3 `AccessDenied` document that `unzip` then reports as a corrupt archive. Without Homebrew:

```
curl -o /tmp/session-manager-plugin.pkg \
  "https://s3.amazonaws.com/session-manager-downloads/plugin/latest/mac_arm64/session-manager-
  plugin.pkg"
sudo installer -pkg /tmp/session-manager-plugin.pkg -target /
sudo ln -sf /usr/local/sessionmanagerplugin/bin/session-manager-plugin /usr/local/bin/session-manager-
plugin
```

`mac_arm64` is Apple Silicon; the plain `mac` path is Intel. **And a fallback worth knowing:** the EC2 console's **Connect** → **Session Manager** tab opens the same shell in a browser tab with nothing installed at all.

Already present: `docker`, `k6`, `node 22`, `pnpm`, `git`.

3.2 Configure the CLI

In the AWS console, go to **IAM** → **Users** → **your user** → **Security credentials** → **Create access key**, choose *Command Line Interface*, and keep the two values on screen while you run:

```
aws configure
# AWS Access Key ID:      <paste>
# AWS Secret Access Key:  <paste>
# Default region name:    ap-south-1
# Default output format: json

aws sts get-caller-identity    # prints your account id if it worked
```

3.3 The names

The product is `exampleprep`, and it lives under the institute's existing `iace.co.in`. One label for the product, one for the environment, one for the service — so every name reads the same way and nothing has to be guessed later.

```
api . staging . exampleprep . iace.co.in
|   |   |   |
service environment product  the zone you already own
```

Production drops the environment label rather than spelling it out, which is the usual convention and keeps the name a reader types most often the shortest.

Thing	Staging	Production (later)
API hostname	<code>api.staging.exampleprep.iace.co.in</code>	<code>api.exampleprep.iace.co.in</code>

Thing	Staging	Production (later)
Student site	<code>staging.examprep.iace.co.in</code>	<code>examprep.iace.co.in</code>
Admin site	<code>admin.staging.examprep.iace.co.in</code>	<code>admin.examprep.iace.co.in</code>
SPA bucket	<code>examprep-staging-spas</code>	<code>examprep-prod-spas</code>
Media bucket	<code>examprep-staging-media</code>	<code>examprep-prod-media</code>
RDS instance	<code>examprep-db</code> (shared)	same instance, second database
Database name	<code>examprep_staging</code>	<code>examprep_production</code>

Everything this platform creates in AWS is named `examprep-`, so a cost report, a cleanup or a second product later is a filter rather than an archaeology exercise. Two of those names can never be changed once created — a **security group's name** and an **IAM role's name** — so they are worth getting right in the next ten minutes rather than living with. A VPC's name is only a tag and is free to change any time.

The buckets carry it for a different reason: an **S3 bucket name is globally unique** — across every AWS account on earth — so a name somebody else has taken is a name you cannot have, and one you have taken is awkward to give up.

One thing deliberately keeps the old name: the metrics. `iace_http_requests_total` and its siblings come from `PREFIX` in `apps/api/src/common/metrics/metrics.service.ts`, and renaming them would orphan every dashboard and alert built on them. The compose project names `iace-api` and `iace-valkey` stay too — they namespace Docker volumes, so changing one would strand the volume holding Caddy's certificates.

| Valkey port | 6380 | 6379 |

Today's shortcut, and it is the right one. Giving the two websites custom subdomains needs a certificate from AWS Certificate Manager in the **us-east-1** region, DNS validation records at GoDaddy, and a CloudFront change — about 40 minutes including waiting. CloudFront hands out a working `https://d1111111abcdef8.cloudfront.net` address with a valid certificate the moment you create it. **Launch on those today**, and add `staging.examprep.iace.co.in` and `admin.staging.examprep.iace.co.in` tomorrow with §12.6. The API subdomain is **not** optional and must be done today, because Caddy cannot get a certificate without it and the student app is an installable PWA that will not register a service worker without real HTTPS.

3.4 A realistic plan for today

30 min	§1–§3	Read, install the CLI, pick the names
45 min	§4	VPC, subnets, security groups, IAM role
20 min	§5	Create RDS — then leave it creating and carry on
20 min	§6	Box B, Valkey up
60 min	§7–§8	Box A, Docker, the environment file, DNS
30 min	§9	Bring the stack up, watch the certificate issue
20 min	§10	Seed the super admin and sign in
45 min	§11	S3, CloudFront, publish both sites
20 min	§12	Verify end to end

Five and a half hours with no surprises; call it a full day with them. **If you run short, stop after §10.** An API with HTTPS and an admin login is a working staging environment — the websites can be published any time after.

4. AWS groundwork

4.1 The VPC and its subnets

Console → **VPC** → **Create VPC**, and choose **VPC and more**, which builds the whole set in one step rather than six.

Field	Value	Why
Name tag	examprep	Everything it creates gets this prefix
IPv4 CIDR	10.0.0.0/16	65,536 addresses; you will use a dozen
IPv6	No	Nothing needs it
Availability Zones	2	RDS refuses a subnet group with fewer
Public subnets	2	Boxes A and B live in the first one
Private subnets	2	RDS lives here
NAT gateways	None	See below. This is a \$32/month decision
VPC endpoints	None	The gateway endpoint for S3 is free but not needed today
DNS hostnames / resolution	Both enabled	RDS hands you a hostname, not an address

Why no NAT gateway. A NAT gateway lets machines in a private subnet reach the internet, and costs about \$32 a month plus traffic. Both of your boxes sit in a *public* subnet with an address of their own, so they reach Docker Hub, Grafana and the SMS provider directly and need no NAT at all. The firewall work is done by security groups instead — which is where it belongs, because a security group travels with the machine and a subnet does not.

Press **Create VPC** and wait for the green bar. Note the VPC id (`vpc-...`) and the id of the first public subnet (`subnet-...` , the one in `ap-south-1a`).

4.2 Two security groups — before any instance exists

Console → **VPC** → **Security groups** → **Create security group**, twice, **in this order**: the second one's rules name the first as their source, so the first has to exist before you can write them. It only has to exist — its own rules can come later.

Set the VPC to `examprep` on both. The dropdown defaults to your *default* VPC, not the one you just created, and this is the field people miss. A security group belongs to exactly one VPC, chosen at creation and **never changeable** — a group in the wrong VPC simply will not appear when you launch the instance, and the fix is to delete it and start again.

Do this before launching anything. If no group exists when you reach the launch wizard, it creates one for you called `launch-wizard-1`, with SSH open to the whole internet in it — the exact rule this design exists to avoid, and easy not to notice. An instance's groups *can* be changed afterwards (Actions → Security → Change security groups, no restart); the group's own name and VPC cannot.

`examprep-api` — Box A.

Direction	Type	Port	Source	Why
Inbound	HTTP	80	0.0.0.0/0	Let's Encrypt's challenge arrives here, and Caddy redirects everything else to 443
Inbound	HTTPS	443	0.0.0.0/0	The actual traffic
Outbound	All	All	0.0.0.0/0	The default; leave it

There is no SSH rule, and that is deliberate. A rule locked to "my IP address" holds right up until your broadband address rotates, and what happens then — at eleven at night, when something is broken — is that somebody

widens it to the whole internet and never narrows it again. You will shell in through SSM Session Manager instead, which needs no inbound rule at all, no key file to lose, and records a transcript of the session.

`examprep-data` — Box B and RDS.

Direction	Type	Port	Source
Inbound	Custom TCP	6379	<code>sg</code> <code>examprep-api</code>
Inbound	Custom TCP	6380	<code>sg</code> <code>examprep-api</code>
Inbound	PostgreSQL	5432	<code>sg</code> <code>examprep-api</code>

In the Source box, type `sg-` and pick `examprep-api` from the dropdown — do not type an address range. That is the whole point: the rule names a *group*, so any box you put in that group is trusted and you never touch this rule again. Only groups in the same VPC appear in that dropdown, which is the first place a VPC mix-up shows itself.

The order for the rest of the day, now that both exist:

```
VPC → examprep-api → examprep-data → instance profile → RDS → Box B → Box A
```

4.3 The instance profile

The boxes need to prove who they are to AWS. Console → **IAM** → **Roles** → **Create role**, trusted entity **AWS service**, use case **EC2**.

This takes two passes, and the wizard does not tell you so. Step 2's "Use existing policy" and "Create inline policy" are a radio pair: pick one and the other's panel disappears. Nothing is being overwritten — you simply cannot do both before the role exists.

Pass one, in the wizard. Leave **Use existing policy** selected and tick these two:

- `AmazonSSMManagedInstanceCore` — this is what makes Session Manager work
- `CloudWatchAgentServerPolicy` — lets the box write logs if you ever turn that on

Search for one, tick it, clear the search and find the other; a ticked policy stays ticked when the filter changes, and the count at the top of the list is the proof. Then **Next**, name the role `examprep-box`, and **Create role**.

Pass two, on the role that now exists. IAM → Roles → `examprep-box` → **Permissions** → **Add permissions** ▾ → **Create inline policy** → **JSON** tab. Paste the whole document below as **one** policy named `examprep-box-inline` — all four statements together, not four policies. Inline policies are keyed by name, so a second one sharing a name replaces the first with no warning. Replace the account id, and stay on the JSON tab: switching to Visual and back can rewrite what you pasted.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "MediaBucket",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::examprep-staging-media",
        "arn:aws:s3:::examprep-staging-media/*"
      ]
    },
    {
      "Sid": "ReadItsOwnEnvironment",
      "Effect": "Allow",
      "Action": ["ssm:GetParameter"],
      "Resource": "arn:aws:ssm:ap-south-1:<ACCOUNT_ID>:parameter/examprep/*"
    }
  ],
}
```

```

{
  "Sid": "DecryptIt",
  "Effect": "Allow",
  "Action": ["kms:Decrypt"],
  "Resource": "*",
  "Condition": {
    "StringEquals": { "kms:ViaService": "ssm.ap-south-1.amazonaws.com" }
  }
},
{
  "Sid": "PullImages",
  "Effect": "Allow",
  "Action": [
    "ecr:GetAuthorizationToken",
    "ecr:BatchGetImage",
    "ecr:GetDownloadUrlForLayer",
    "ecr:BatchCheckLayerAvailability"
  ],
  "Resource": "*"
}
]
}

```

Both boxes will use this role. Check what actually landed, because the console view is easy to misread:

```

aws iam list-attached-role-policies --role-name examprep-box # the two managed ones
aws iam list-role-policies --role-name examprep-box          # exactly one inline name
aws iam get-role-policy --role-name examprep-box --policy-name examprep-box-inline

```

Note what is NOT in it: `ssm:PutParameter`. A box reads its environment and cannot overwrite it, so an attacker who gets onto the box cannot poison what every replacement will pull afterwards. Pushing is an admin action from your laptop (§6.3), and `put-parameter` run on a box is *supposed* to be denied.

Why this list and nothing more. The media bucket because the API uploads and signs question images. SSM and KMS because the environment file's master copy lives there encrypted. ECR because later you will build images in CI and pull them here. `AmazonSSMManagedInstanceCore` because that is your shell. Nothing else — a role is a standing permission, and a standing permission is the thing an attacker inherits.

4.4 An Elastic IP

Console → **EC2** → **Elastic IPs** → **Allocate Elastic IP address** → **Allocate**. Do not associate it with anything yet. Write the address down; GoDaddy needs it in §8 and it costs \$3.65 a month whether or not it is attached.

5. RDS — the database

Create this early and leave it building; it takes 10–15 minutes and nothing else waits on it.

Console → **RDS** → **Create database**.

Field	Value	Why
Method	Standard create	Easy create picks sizes you do not want
Engine	PostgreSQL	
Version	17, the highest 17.x offered	Not 18, and not 16 — see below
Template	Free tier if the account is new, otherwise Dev/Test	
DB instance identifier	<code>examprep-db</code>	One instance, a database per environment
Master username	<code>postgres</code>	
Credentials management	Self managed	See below
Master password	<code>openssl rand -base64 36</code>	Not the password the app uses
Instance class	db.t4g.micro	2 vCPU, 1 GB. Not <code>db.t3.micro</code> — same machine, \$18.98 against \$15.33
Storage	gp3, 20 GB , autoscaling on to 100 GB	gp3 gives 3,000 IOPS at any size
Multi-AZ	No	Doubles the price for availability staging does not need
Compute resource	Don't connect to an EC2 compute resource	See below
VPC	<code>examprep</code>	
Subnet group	Create new — <code>examprep-db-private</code> , the two private subnets only	Never "Automatic setup"; see below
Public access	No	
Security group	<code>examprep-data</code>	
Availability Zone	<code>ap-south-1a</code>	Same zone as the boxes, so traffic is free and fast
Database authentication	Password	Not IAM — Prisma takes one static URL and has no hook for a per-connection credential, so a 15-minute IAM token breaks the pool as it recycles, an hour in and intermittently. Additive and reversible later
Initial database name	<code>examprep_staging</code>	Leave this blank and RDS creates no database at all, which is a confusing first error
Backup retention	7 days	This is point-in-time restore. Free up to your allocated storage
Backup window	<code>19:00–19:30 UTC</code>	00:30–01:00 IST, nobody is sitting an exam
Auto minor version upgrade	On	Security fixes land without anyone remembering
Maintenance window	<code>Sun 20:00–20:30 UTC</code>	That upgrade is a brief restart. RDS refuses an overlap with the backup window, and left on the default AWS picks a slot without consulting your exam calendar

Field	Value	Why
Encryption	On, default key	Free
Performance Insights	On, 7 days	Free at that retention, and it is how you see a slow query
Deletion protection	On	

Why 17 and not 18, now that 18 is on the list. The deciding reason is in the repo rather than in AWS: `docker-compose.yml` pins `postgres:17-alpine`, so every unit test, every `test:db` run and every migration ever applied has run against 17, and Prisma 6.19.3 is validated there. Choosing 18 would make staging the only place 18 is ever exercised, so the first difference surfaces during a demo instead of in CI. The reason 17 beat 16 still holds too: once a version leaves standard support, RDS Extended Support costs \$0.114 per vCPU-hour, more than the instance — and 17 runs to **28 February 2030**, which is longer than this schema will live. Take the **highest 17.x** on offer: minors are backward-compatible and upgrade in place, so starting at the top buys fewer forced restarts. Moving to 18 later is a major upgrade — in place with downtime, or Blue/Green with almost none — and nothing here forecloses it.

Don't let it connect you to an EC2 instance either. Box A does not exist yet — that is the point of starting RDS first — but the real reason is that the shortcut creates its own pair of security groups (`rds-ec2-1` and `ec2-rds-1`), attaches one to the database and the other to your instance, and wires those together. You would have that pair sitting beside `examplep-api` and `examplep-data`, two mechanisms doing one job, and no obvious answer later to which rule is load-bearing. What §4.2 built is strictly better: `examplep-data` admits 5432 from the *group*, so any box joining it is trusted without anyone touching RDS again. The shortcut wires instance-to-instance and cannot express that.

Never let it create the subnet group automatically. Automatic setup puts every subnet in the VPC into the group, public ones included, and RDS then places the instance — and any later replacement or Multi-AZ standby — anywhere in that set. `Public access: No` still means no public address, so it would not be reachable; but the database would be sitting in a subnet with a route to the internet gateway, with the security group as the only thing left. That layer is free to keep. Tick both availability zones first: the subnet list only shows subnets in the zones you have selected, so otherwise half of them never appear. Changing the subnet group afterwards is supported but disruptive — an outage, and possibly a new private address.

Why `micro` is enough, and what actually moves it. Not CPU and not traffic — **connections**. RDS derives `max_connections` from memory: roughly 112 on 1 GB, 225 on 2 GB. The pools the app opens are `exam 8 + core 25 + worker 25 = 58` for one environment, plus 10 for the other. The thing that exhausts a `micro` is adding a second `exam` container, not adding students.

Self managed, and why. The other option hands the master password to Secrets Manager at \$0.40 a month. What it manages is the **master** credential only — not the application's, which is what people assume. The app connects as `examplep_staging` with a password in `deploy/.env`, and Secrets Manager never sees it. So the thing being managed is a credential used about three times ever: once below to create the application role, and then only to break glass. Put it in the institute's password manager, and keep a copy where every other secret here lives — SSM, free, the same mechanism as §11:

```
openssl rand -base64 36
aws ssm put-parameter --name /examplep/rds/master --type SecureString --value '<the password>'
```

If there is no institute password manager, so the real alternative is a text file on a laptop, then Secrets Manager is the honest answer and forty cents is cheap. Either way it is **reversible** — RDS can be modified later to take master credential management over, or hand it back.

5.1 What you cannot change afterwards

Most of this form is a Modify away from being something else. Four things are not, and the first is the one people hit:

Fixed at creation	Changeable later
The VPC	Instance class — brief outage
The master username	Storage size, up only and online
The initial database name	Storage type
Encryption at rest	Subnet group — <i>same VPC only</i> , and disruptive
	Security groups — immediate, no outage
	Multi-AZ on or off
	Public accessibility
	Backup retention and both windows
	Master password, and who manages it
	Minor and major engine version
	Deletion protection
	The instance identifier, though the endpoint changes with it

Encryption is the one to check twice before pressing Create. It cannot be switched on later — the fix is snapshot, restore to an encrypted instance, repoint — and it is free, so there is no reason for it to be off.

If the VPC is wrong, delete and start again. With no data in it that is fifteen minutes, not a migration: Modify → uncheck Deletion protection → Apply immediately, then Actions → Delete with no final snapshot. Once it holds data the move is snapshot → Restore snapshot into the right subnet group, which gives you a new instance and a new endpoint to repoint `DATABASE_URL` at.

5.2 The application's database user

When the instance says **Available**, copy its endpoint — something like `examprep-db.cxabc123.ap-south-1.rds.amazonaws.com`. You cannot reach it from your laptop; it has no public address, by design. So run this from Box A once that exists (\$7.5), or from any machine inside the VPC:

```
-- as the master user: psql "postgresql://postgres:MASTER@<endpoint>:5432/examprep_staging"
CREATE USER examprep_staging WITH PASSWORD 'a-long-random-password' CONNECTION LIMIT 25;
GRANT ALL ON SCHEMA public TO examprep_staging;
GRANT ALL PRIVILEGES ON DATABASE examprep_staging TO examprep_staging;
ALTER ROLE examprep_staging SET statement_timeout = '30s';
ALTER ROLE examprep_staging SET idle_in_transaction_session_timeout = '60s';
```

Why those two timeouts are set on the role. Staging and production share this instance. A runaway query on staging would otherwise hold CPU and locks that production needs, and a transaction left open by a crashed debugging session would hold them for ever. `CONNECTION LIMIT` is the hard stop: staging cannot take more than 25 of the instance's ~112 however many containers somebody starts.

6. Box B — Valkey

6.1 Launch it

Console → **EC2** → **Launch instance**.

Field	Value
Name	examprep-valkey
AMI	Amazon Linux 2023, architecture 64-bit (Arm)
Instance type	t4g.micro
Key pair	Proceed without a key pair
VPC / subnet	examprep, the public subnet in ap-south-1a
Auto-assign public IP	Enable
Security group	existing → examprep-data
Storage	10 GiB gp3
Advanced → IAM instance profile	examprep-box
Advanced → Metadata version	V2 only (token required)
Advanced → Metadata response hop limit	2

The architecture is the trap. `t4g` is Graviton, which is ARM. Pick an x86 AMI with it and the instance will not boot, with an error that does not say why. The AMI dropdown has an architecture selector — set it before you choose.

Why "proceed without a key pair". There is no SSH rule to use it with. Session Manager is the shell.

Why the hop limit is 2. The instance metadata service hands out your role's credentials at `169.254.169.254`. IMDSv2 requires a PUT to get a token first, which blocks the one-request server-side-request-forgery attack that would otherwise walk off with the role. The hop limit is how many network hops the response may travel: the default 1 works from the host, but a process inside a Docker container is one hop further out and gets nothing. At 1 your containers silently cannot reach S3.

Why it has a public address at all. Only so it can pull the Valkey image from Docker Hub. Nothing on the internet has a rule that reaches it — `examprep-data` only admits the API group.

6.2 Get a shell, install Docker

```
aws ssm start-session --target i-0123456789abcdef0
```

If that says the target is not connected, wait two minutes — the SSM agent registers shortly after boot — and confirm the instance profile is attached.

```
sudo dnf update -y
sudo dnf install -y docker git
sudo systemctl enable --now docker
sudo usermod -aG docker ssm-user
exit
```

`exit` ends the session; that is what you want. Start a new one so the group membership applies, then check **both** halves:

```
docker --version && docker compose version
```

The second one will fail, and the runbook needs it on the very next line. Amazon Linux 2023's `docker` package does not ship the Compose v2 plugin and it is not in the repositories either. Install it by hand:

```
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-aarch64 -
o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
docker compose version
```

`aarch64`, **not** `x86_64` — these are Graviton boxes. The x86 binary downloads perfectly and then fails with "cannot execute binary file", which reads like a corrupt download rather than the wrong architecture.

6.3 Bring Valkey up

Do not clone the repository on this box. It is private, so cloning means putting a GitHub credential on a machine whose only job is to run one container — and that container needs exactly two files. Write them instead. Box A is different and does clone (\$7.4), because it builds the image from source.

```
sudo mkdir -p /opt/examprep/deploy/valkey
sudo chown -R ssm-user:ssm-user /opt/examprep
cd /opt/examprep/deploy/valkey
```

Own the directory first, and then use a plain redirect. `sudo cat > file` does not work: the `>` is handled by your shell before `sudo` runs, so the file is opened as you regardless and you get permission denied on a line that looks like it should have worked. The alternative, if a path is not yours, is `sudo tee file > /dev/null <<'EOF'`, where `tee` itself runs privileged.

```
cat > compose.yml <<'YAML'
name: iace-valkey

services:
  valkey-prod:
    image: valkey/valkey:8-alpine
    restart: unless-stopped
    command:
      - valkey-server
      - --appendonly
      - 'yes'
      - --maxmemory-policy
      - noeviction
      - --maxmemory
      - ${PROD_MAXMEMORY:-1200mb}
      - --requirepass
      - ${VALKEY_PROD_PASSWORD:?set it in .env}
    environment:
      REDISCLI_AUTH: ${VALKEY_PROD_PASSWORD}
    ports:
      - '${PRIVATE_IP}:6379:6379'
    volumes:
      - prod-data:/data
    stop_grace_period: 60s
    healthcheck: &ping
      test: ['CMD', 'valkey-cli', 'ping']
      interval: 10s
      timeout: 5s
      retries: 10

  valkey-staging:
    image: valkey/valkey:8-alpine
    restart: unless-stopped
    command:
```

```

- valkey-server
- --appendonly
- 'yes'
- --maxmemory-policy
- noeviction
- --maxmemory
- ${STAGING_MAXMEMORY:-256mb}
- --requirepass
- ${VALKEY_STAGING_PASSWORD:?set it in .env}
environment:
  REDISCLI_AUTH: ${VALKEY_STAGING_PASSWORD}
ports:
  - '${PRIVATE_IP}:6380:6379'
volumes:
  - staging-data:/data
stop_grace_period: 60s
healthcheck: *ping

volumes:
  prod-data:
  staging-data:
YAML

```

Keep `name: iace-valkey` **exactly as it is**. It namespaces the Docker volumes holding the AOF, so changing it strands the data. `deploy/valkey/compose.yml` in the repository is the source of record if this ever drifts; the only reason it is reproduced here is that this box has no checkout.

`REDISCLI_AUTH` is why the healthcheck works at all: `valkey-cli ping` answers `NOAUTH` the moment `requirepass` is set, so without it every container reports unhealthy forever. Putting it in the environment rather than on the healthcheck command keeps the password out of `ps` output.

Find this box's **private** address — the `10.0.x.x` one, not the public:

```

TOKEN=$(curl -sX PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 60")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/local-ipv4; echo

```

Now the environment file. Generate the two passwords first, as two separate runs:

```
openssl rand -base64 36
```

```
openssl rand -base64 36
```

Both are needed even though only staging runs today — compose's `:?` guard refuses an empty one rather than starting a Valkey with no password.

```

cat > .env <<'ENV'
PRIVATE_IP=10.0.0.32
VALKEY_PROD_PASSWORD=
VALKEY_STAGING_PASSWORD=
PROD_MAXMEMORY=1200mb
STAGING_MAXMEMORY=256mb
ENV
vi .env

```

Put your own private address in and paste a password onto each line. Compose reads this file from beside `compose.yml` with no flag, so a password never reaches shell history. Then lock it and start:

```
sudo chown root:root .env && sudo chmod 600 .env
sudo docker compose up -d valkey-staging
sudo docker compose ps
```

Everything from here needs `sudo`, including `vi`, because the file is now `root:root 600`. Opening it without `sudo` gives you a read-only buffer, and `vi` only says so when you try to save.

You want `Up ... (healthy)`. If it says `unhealthy`, read §22.

Three locks, and the third is the one that matters. The security group keeps the internet off it. `PRIVATE_IP` in the compose file publishes the port on the VPC address only, so even a wrong security group does not put Valkey on a public interface. And `requirepass` is the one that still holds when somebody edits the group wrongly. What is behind those locks is every live sitting, every session token and every OTP, on port 6379 — which is among the most-scanned ports on the internet.

Prove the locks rather than assuming them. The second and third only show themselves under test:

```
sudo ss -ltnp | grep 638
sudo docker compose exec -T valkey-staging valkey-cli config get maxmemory-policy
sudo docker run --rm --network host valkey/valkey:8-alpine valkey-cli -h 10.0.0.32 -p 6380 ping
```

The listener must be on `10.0.0.32:6380` and not `0.0.0.0`, the policy must read `noeviction`, and that last ping must answer `NOAUTH Authentication required`. A `PONG` there means the password is not set and the stack came up anyway.

Push the file to SSM from your LAPTOP, not from the box. The instance profile carries `ssm:GetParameter` and nothing else (§4.3), so `put-parameter` on the box is denied — and that is the design rather than a gap: **a box pulls its secrets and must not be able to overwrite them**. A credential that can rewrite its own source is one an attacker on that box can poison for every replacement that follows.

So, on your laptop, with the two passwords you generated:

```
cat > /tmp/valkey.env <<'ENV'
PRIVATE_IP=10.0.0.32
VALKEY_PROD_PASSWORD=
VALKEY_STAGING_PASSWORD=
PROD_MAXMEMORY=1200mb
STAGING_MAXMEMORY=256mb
ENV
vi /tmp/valkey.env
aws ssm put-parameter --region ap-south-1 --name /examprep/valkey/env \
  --type SecureString --overwrite --value file:///tmp/valkey.env
rm /tmp/valkey.env
```

The box's own `ssm:GetParameter` is what brings it back on a replacement, which is the half that has to work under pressure.

7. Box A — the API box

7.1 Launch it

Same as §6.1 with these differences:

Field	Value	Why
Name	<code>examprep-api-staging</code>	
Instance type	t4g.medium (2 vCPU, 4 GB)	You build the image on the box today, and the install wants it — see below. Drop to <code>small</code> later, once images come from ECR
Security group	<code>examprep-api</code>	
Storage	30 GiB gp3	Docker images, build cache, logs

Everything else — ARM AML, no key pair, public subnet in `1a`, instance profile `examprep-box`, IMDSv2 required, hop limit 2 — is identical.

Nothing of the front end is built here, and that is worth being clear about. The image copies only `apps/api`, `packages` and `prisma`, and runs `pnpm --filter @iace/api... build` — the API plus the two workspace packages it depends on, `@iace/contracts` and `@iace/config`. Neither SPA is in the image at all. **The two Vite builds happen wherever you run** `deploy/publish-spas.sh`, which is your laptop today and CI later (§11.5).

What actually wants the 4 GB is `pnpm install`, and it installs more than the API needs: `apps/api/Dockerfile` copies **every** workspace manifest before installing, so the lockfile stays frozen and verifiable — and the side effect is that the install resolves Expo, React Native and both SPA toolchains as well. Add Prisma's engine downloads and `tsc` on top.

There is a real saving available later, when images move to CI and ECR:

```
RUN --mount=type=cache,id=pnpm,target=/pnpm/store \
  pnpm install --frozen-lockfile --filter @iace/api...
```

That skips the mobile and SPA toolchains entirely and changes nothing in the runtime image, since `pnpm deploy --filter @iace/api` already prunes to the API. It needs a real build to verify the filter still resolves against the frozen lockfile, so it is a change for the day you move the build off the box — not for the day you are trying to get one booted.

7.2 Attach the Elastic IP

Console → **EC2** → **Elastic IPs** → select the address from §4.4 → **Actions** → **Associate** → choose this instance → **Associate**.

7.3 Shell in and install Docker

Exactly as §6.2.

7.4 Get the code

```
sudo mkdir -p /opt/examprep && sudo chown ssm-user /opt/examprep
cd /opt/examprep
git clone https://github.com/<your-org>/iace.git .
```

A private repository needs a credential. The least bad option for today is a **GitHub deploy key** — read-only, repo-scoped, and revocable:

```
ssh-keygen -t ed25519 -f ~/.ssh/examprep_deploy -N ""
cat ~/.ssh/examprep_deploy.pub
# paste into GitHub → the repo → Settings → Deploy keys → Add, read-only
git clone git@github.com:<your-org>/iace.git .
```

7.5 Create the database user

`psql` is not installed and you only need it once:

```
sudo dnf install -y postgresql17
psql "postgresql://postgres:MASTER_PASSWORD@examprep-db.xxxx.ap-south-1.rds.amazonaws.com:5432/examprep_staging"
```

Run the SQL from §5.2. If it hangs instead of refusing, the security group is wrong — RDS must admit 5432 from `examprep-api`.

7.6 The environment file

```
cd /opt/examprep
cp deploy/.env.example deploy/.env
openssl rand -base64 48
vi deploy/.env
```

Run that `openssl` line **four separate times** — one each for `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `PIN_PEPPER` and `METRICS_TOKEN`. Do not put the explanation after the command as a `#` comment: `zsh` does not treat `#` as a comment in an interactive shell unless `interactive_comments` is set, so a pasted line arrives at the program with the comment attached and fails with something unhelpful like `rand: Extra option`.

Every value, and where it comes from:

Variable	Value	Notes
<code>API_HOST</code>	<code>api.staging.examprep.iace.co.in</code>	Must match §8 exactly, or no certificate
<code>ACME_EMAIL</code>	<code>developer@iace.co.in</code>	Where Let's Encrypt sends expiry warnings
<code>NODE_ENV</code>	<code>development</code>	Deliberate. See below
<code>OTP_SENDER</code>	<code>console</code>	The code comes back in the HTTP response as <code>devCode</code>
<code>DATABASE_URL</code>	<code>postgresql://examprep_staging:PASS@<endpoint>:5432/examprep_staging?schema=public&connection_limit=25&pool_timeout=20</code>	<code>connection_limit</code> is not optional
<code>REDIS_URL</code>	<code>redis://:VALKEY_STAGING_PASSWORD@10.0.1.20:6380</code>	Box B's private address. Note the empty username before the colon
<code>JWT_ACCESS_SECRET</code> etc.	the four generated values	
<code>S3_REGION</code>	<code>ap-south-1</code>	

Variable	Value	Notes
<code>S3_BUCKET</code>	<code>examprep-staging-media</code>	Created in §11
<code>S3_ACCESS_KEY_ID</code> <code>/ SECRET</code>	leave empty	The instance profile signs. The API refuses half a pair
<code>MEDIA_BASE_URL</code>	the media CloudFront domain, and nothing after it	§11.3 gives it to you; come back and fill it in
<code>CORS_ORIGINS</code>	the two CloudFront domains, comma-separated	§11.3. Until then the sites cannot call the API
<code>TRUST_PROXY_HOPS</code>	<code>1</code>	Caddy is the one hop. At 0 every caller shares one rate-limit bucket and the whole hall locks itself out
<code>ENVIRONMENT</code>	<code>staging</code>	The <code>env</code> label on every log line
<code>LOKI_*</code> / <code>PROM_*</code>	§14	Leave blank for now; Alloy will just fail to ship and nothing else breaks

Why `NODE_ENV=development` **on a deployed box.** In production mode the API refuses to start unless it can send a real SMS, because a student signs in with an SMS OTP and an admin with an emailed one — and the DLT registration behind that SMS does not exist yet (§21). In development mode `OTP_SENDER=console` returns the code in the HTTP response as `devCode`, so a tester signs in with no provider at all. `CORS_ORIGINS` is still enforced either way. Switch both to production when the SMS vendor is live.

Lock it down:

```
sudo chown root:root deploy/.env && sudo chmod 600 deploy/.env
```

Then back it up **from your laptop**, for the reason in §6.3 — the instance profile can read a parameter and not write one, so `put-parameter` on the box is denied by design. Keep the filled-in file locally, strip the comments and push that:

```
grep -v '^#' env.staging | grep . > /tmp/env.stripped
aws ssm put-parameter --region ap-south-1 --name /examprep/staging/env \
  --type SecureString --overwrite --value file:///tmp/env.stripped
rm /tmp/env.stripped
```

The comments are stripped because a Standard-tier parameter caps at **4 KB** and the commented file is close to it. Nothing reads this at runtime — it is there so that replacing a dead box is one command instead of an evening.

8. DNS at GoDaddy

Sign in at GoDaddy → **My Products** → **Domains** → `iace.co.in` → **DNS** → **Manage Zones**.

8.1 The one record you must have today

Add → **A**

Field	Value
Type	<code>A</code>
Name	<code>api.staging.examprep</code>
Value	your Elastic IP, e.g. <code>13.234.56.78</code>
TTL	Custom → 600 seconds

The Name field is relative to the zone. You own `iace.co.in`, so typing `api.staging.examprep` creates `api.staging.examprep.iace.co.in`. Typing the full name creates `api.staging.examprep.iace.co.in.iace.co.in`, which is the single most common mistake here and produces a certificate failure that looks like a Caddy bug.

Check it from your laptop before going further:

```
dig +short api.staging.examprep.iace.co.in
# must print your Elastic IP and nothing else
```

GoDaddy usually propagates in a minute or two. **Do not start Caddy until this prints the right address** — Let's Encrypt rate-limits failed authorisations, and five failures for the same hostname in an hour locks you out for an hour.

8.2 The websites get no Elastic IP and no A record — ever

This is the question that trips everybody. The API box has an address of its own, so it takes an **A** record. The two websites are not on a box at all: they are files in S3 served by CloudFront, which is not a server but a few hundred edge locations. DNS hands each visitor the addresses nearest to them — a different answer in Mumbai and in Delhi — and those addresses change without notice. An A record would pin every student to one edge that may not exist next month.

So a website takes a **CNAME** pointing at the distribution's own name, and AWS resolves it per visitor.

	Backend	Frontend
What runs it	One EC2 box	CloudFront edges worldwide
Address	An Elastic IP, yours, fixed	None you can see or pin
DNS record	<code>A</code> → <code>13.234.56.78</code>	<code>CNAME</code> → <code>d111111abcdef8.cloudfront.net</code>
Certificate	Caddy, from Let's Encrypt	ACM in us-east-1, free
Where TLS ends	Caddy, on your box	At the edge nearest the student

Which is also why the websites need no DNS today at all. Each distribution arrives with `https://d111111abcdef8.cloudfront.net` and a valid certificate already on it. The API subdomain is the one that cannot wait, because Caddy has nothing to ask Let's Encrypt for without it.

8.3 The website records — after §11, optional today

Once CloudFront gives you its domains, come back and add two CNAMEs:

Type	Name	Value	TTL
CNAME	staging.examprep	d111111abcdef8.cloudfront.net	600
CNAME	admin.staging.examprep	d222222abcdef8.cloudfront.net	600

These alone are **not enough**, and the failure is confusing: a CloudFront distribution rejects any `Host` header it has not been told about, and answers `403 The request could not be satisfied` — which reads like a permissions problem rather than a missing setting. §12.6 is the other half: a certificate in us-east-1 and that hostname added to the distribution as an alternate domain name. Until both halves are done, use the `*.cloudfront.net` addresses, which work perfectly.

You cannot do this for the bare domain. DNS forbids a CNAME at a zone apex, and GoDaddy's free DNS has no ALIAS or ANAME to work around it. If `iace.co.in` itself ever has to point at CloudFront, that is the moment the zone moves to Route 53, whose ALIAS record does exactly this and is free. Every name in this document is a subdomain, so it does not arise today.

8.4 What not to touch

Leave the existing MX, TXT and `www` records alone. They are the institute's email and main website. You are only adding subdomains.

9. Bring the stack up

```
cd /opt/examprep
sudo docker compose -f deploy/compose.yml --env-file deploy/.env up -d --build
```

The first build takes **8–15 minutes** on a `t4g.medium` — it installs the whole workspace and builds the API twice, once for the runtime image and once for the migration image. Watch it:

```
sudo docker compose -f deploy/compose.yml --env-file deploy/.env ps
sudo docker compose -f deploy/compose.yml --env-file deploy/.env logs -f
```

What a healthy stack looks like:

Service	State
migrate	Exited (0) — it ran the migrations and stopped. This is correct, not a failure
exam, core, worker	Up ... (healthy)
caddy	Up
alloy	Up — or restarting, if you left the Grafana values blank

The order is enforced, not hoped for. The three API containers declare `depends_on: migrate: condition: service_completed_successfully`, so Docker will not start them until the migration container has exited zero. A release can never serve traffic against a schema that has not been migrated.

9.1 Watch the certificate issue

```
sudo docker compose -f deploy/compose.yml --env-file deploy/.env logs caddy | grep -i certificate
```

You are looking for `certificate obtained successfully`. From your laptop:

```
curl -i https://api.staging.examprep.iace.co.in/health
```

A JSON body and `HTTP/2 200` means the whole chain works: DNS, the security group, Caddy, TLS, and the `core` container behind it.

If the certificate does not issue, it is almost always one of three things, in this order of likelihood: DNS has not propagated (`dig` again); port 80 is not open to the world in `examprep-api` (the HTTP-01 challenge arrives on 80, not 443); or `API_HOST` in the env file does not exactly match the record you created.

9.2 The two health endpoints, and why there are two

```
curl -s https://api.staging.examprep.iace.co.in/health | jq
curl -s https://api.staging.examprep.iace.co.in/health/ready | jq
```

`/health` answers 200 as long as the process is alive, even when a dependency is down. `/health/ready` probes Postgres, Valkey, S3 and the queue and fails if any of them is unreachable — and it is the one the container healthcheck uses. The distinction matters during an incident: a box that is up but cannot reach Valkey should be visibly not-ready, not quietly answering requests it will fail.

10. Seed the first super admin

Migrations create tables and no rows. Admins cannot self-register and nothing in the application creates one, so **until you do this there is no way into the admin app at all.**

From Box A:

```
cd /opt/examprep
sudo docker compose -f deploy/compose.yml --env-file deploy/.env run --rm \
  --entrypoint sh migrate -c 'npx prisma db execute --file prisma/seed.sql --schema
  prisma/schema.prisma'
```

That inserts the super admin `developer@iace.co.in`, the eleven branches, the SSC CGL exam with its two tiers, and the Tier 1 base configuration with its four sections. It is idempotent — every insert is guarded, so running it twice changes nothing.

To use a different address instead, connect with `psql` and:

```
INSERT INTO "Admin" (id, email, "isSuperAdmin", "isActive", "allBranches")
VALUES (gen_random_uuid()::text, 'you@iace.co.in', true, true, true);
```

The ONLINE branch is not optional. A student's branch decides which standard series they can reach, so a student with no branch sees only what a free series, a program, an event or an explicit grant opens — however many exams they are enrolled on. The seed creates it and the API refuses to duplicate, rename, retire or delete it.

10.1 Sign in

The admin site is not published yet, so prove the API works directly:

```
curl -s -X POST https://api.staging.examprep.iace.co.in/auth/admin/otp/request \
  -H 'content-type: application/json' -H 'x-client: WEB' \
  -d '{"email":"developer@iace.co.in"}' | jq
```

Because `OTP_SENDER=console`, the response carries `devCode`. That is the whole point of running staging in development mode: no email provider, no SMS vendor, and a tester can still sign in.

11. The websites — S3 and CloudFront

Both single-page apps are just files. There is no server to run.

11.1 Two buckets

Console → **S3** → **Create bucket**, twice. Region `ap-south-1`, **Block all public access ON** for both. Nothing is ever public directly; CloudFront reaches in through an origin access control.

- `exampleprep-staging-spas` — the two websites
- `exampleprep-staging-media` — question images and uploads

11.2 Three distributions

Console → **CloudFront** → **Create distribution**, three times.

Student site

Field	Value
Origin domain	<code>exampleprep-staging-spas.s3.ap-south-1.amazonaws.com</code>
Origin path	<code>/student</code>
Origin access	Origin access control → Create new → then Copy policy and paste it into the bucket's permissions
Viewer protocol policy	Redirect HTTP to HTTPS
Allowed methods	GET, HEAD
Compress objects	Yes
Default root object	<code>index.html</code>
Price class	Use only North America, Europe, Asia — or India-only if offered

Admin site — identical but origin path `/admin`.

Media — origin `exampleprep-staging-media.s3.ap-south-1.amazonaws.com`, no origin path, same OAC treatment.

Why the price class matters. The default includes every edge location on earth, including the expensive ones. Your students are in India.

11.3 Write down the three domains

Each distribution shows a **Distribution domain name** like `d111111abcdef8.cloudfront.net`. Go back to `deploy/.env` on Box A and fill in:

```
MEDIA_BASE_URL=https://dmedia111111.cloudfront.net
CORS_ORIGINS=https://d111111abcdef8.cloudfront.net,https://d222222abcdef8.cloudfront.net
```

No bucket name in `MEDIA_BASE_URL`, and this one is easy to get wrong. The API builds an image URL as `${MEDIA_BASE_URL}/${key}`, and that distribution's origin is the media bucket — so the key alone is the path. Put the bucket in as well and CloudFront asks S3 for `exampleprep-staging-media/questions/images/...`, which does not exist, and **every question image is a 404**. The local `.env.example` does carry the bucket, because MinIO is path-style; CloudFront is not.

Then restart so the API picks them up:

```
sudo docker compose -f deploy/compose.yml --env-file deploy/.env up -d
```

`CORS_ORIGINS` is the one that bites. A browser refuses to let a page on one origin call an API on another unless the API says it may. Get this wrong and the site loads perfectly and every single request fails, with an error only visible in the browser console. The values must be exact — scheme included, no trailing slash.

11.4 The single-page-app routing rule

A single-page app has routes like `/tests/abc/attempt` that do not exist as files. Without help, CloudFront asks S3 for that key, S3 says 404, and the student sees an error page.

Do not fix this with the "custom error response 404 → `/index.html` → 200" trick. It works for page routes and also turns genuinely missing files into HTML — so a JavaScript chunk that failed to upload is served as HTML, and the browser reports a syntax error in a file that looks fine.

Instead: **CloudFront → Functions → Create function**, runtime `cloudfront-js-2.0`:

```
function handler(event) {
  var request = event.request;
  var uri = request.uri;
  // A path with a dot in its last segment is a real file: let S3 answer, 404 included.
  if (uri.indexOf('.') === -1) request.uri = '/index.html';
  return request;
}
```

Publish it, then on both website distributions: **Behaviors → Default → Edit → Function associations → Viewer request → CloudFront Functions** → your function.

11.5 Publish

From your laptop, not the box — it builds the two sites:

```
cd ~/Projects/iace
./deploy/publish-spas.sh https://api.staging.examprep.iace.co.in examprep-staging-spas E111111ABCDEF
E222222ABCDEF
```

The two `E...` values are the **distribution IDs**, from the CloudFront list — not the domains.

Read the script once. Three things in it are not obvious:

- `VITE_API_URL` is baked in at compile time. The API address is not a runtime setting for the websites; it is compiled into the JavaScript. An environment is therefore a *build*, and a staging bundle can never be promoted to production — it would call the staging API.
- **Hashed assets upload first, the shell second.** The other order serves, for a few seconds, an `index.html` that references chunks which are not uploaded yet — and every visitor in that window gets a blank page.
- **Old asset files are never deleted.** A tab opened before the deploy still lazy-loads chunks under their old hashed names. Keeping them costs about a cent a year.

12. Verify, end to end

12.1 The checklist

```
# 1. The API is up and all dependencies are reachable
curl -s https://api.staging.examprep.iace.co.in/health/ready | jq

# 2. The certificate is real and not self-signed
curl -sI https://api.staging.examprep.iace.co.in/health | head -1

# 3. Caddy routes to the right container
curl -s -o /dev/null -w '%{http_code}\n' https://api.staging.examprep.iace.co.in/me/leaderboard #
exam
curl -s -o /dev/null -w '%{http_code}\n' https://api.staging.examprep.iace.co.in/auth/me #
core
# 401 from both is correct – unauthenticated, but it reached a container

# 4. Metrics answer with the token, and refuse without it
curl -s -o /dev/null -w '%{http_code}\n' https://api.staging.examprep.iace.co.in/metrics
curl -s -H "authorization: Bearer $METRICS_TOKEN" https://api.staging.examprep.iace.co.in/metrics |
head -5

# 5. The websites load
open https://d1111111abcdef8.cloudfront.net
open https://d2222222abcdef8.cloudfront.net
```

12.2 The real test

Sign in to the admin site with `developer@iace.co.in`, read the `devCode` out of the API response in the browser's network tab, create one student, create one test, publish it. Then sign in to the student site as that student and start the test. That is the whole product exercised, and it is worth the fifteen minutes.

12.3 Tell the tester

They need three things: the student URL, the admin URL, and the fact that **the OTP is in the network response, not in any SMS** — the field is `devCode` on the OTP request call. Without that last sentence they will sit waiting for a text message.

12.4 Snapshot the box

Console → **EC2** → **the instance** → **Actions** → **Image and templates** → **Create image**. Name it `examprep-api-staging-<date>`. It takes a few minutes and costs pennies, and a new box from this image needs only the env file from SSM — which is a three-minute recovery instead of repeating this whole document.

12.5 What you have, and what you do not

You have HTTPS, a database with backups, a cache, telemetry, two published sites, and an admin who can sign in. You do **not** have: SMS, real email, custom domains on the two sites, or a second box. None of those stop a tester.

12.6 Custom domains for the sites — tomorrow, 40 minutes

1. **ACM** → **us-east-1** → **Request certificate** for `staging.examprep.iace.co.in` and `admin.staging.examprep.iace.co.in`. It must be us-east-1; CloudFront reads certificates from nowhere else, whatever region your buckets are in.
2. ACM gives you one CNAME per name. Add them at GoDaddy, exactly as given, with the `.iace.co.in` suffix stripped from the Name field.
3. Wait for **Issued** — usually minutes.
4. On each distribution: **Edit** → **Alternate domain name (CNAME)** → the hostname → **Custom SSL certificate** → the ACM one → Save, and wait for Deployed.

5. Add the two CNAMEs from §8.2.

6. Add the new origins to `CORS_ORIGINS` on the API and restart it. The `cloudfront.net` ones can stay alongside.

No rebuild is needed, and this is worth understanding. `VITE_API_URL` is the only compile-time variable either app has (`apps/exams/src/lib/api.ts`, `apps/admin/src/lib/api.ts`), and it holds the **API's** address, not the site's own. Moving a site to a new hostname changes nothing inside its bundle. What does have to change is `CORS_ORIGINS` — miss it and the site loads perfectly while every request fails, visible only in the browser console.

13. Caddy — what it is, and every line of the Caddyfile

Caddy is the only thing on Box A that listens to the internet. It does three jobs: it gets and renews the HTTPS certificate, it terminates TLS, and it forwards each request to whichever container should answer it.

13.1 The file, annotated

deploy/Caddyfile:

```
{
    email {$ACME_EMAIL}
}
```

The global block. One setting: the address Let's Encrypt writes to if a renewal is failing. `{$VAR}` is Caddy's syntax for an environment variable, and compose passes it in.

```
{$API_HOST} {
```

A **site block**. The hostname is the configuration — Caddy sees a real hostname and automatically obtains a certificate for it, redirects port 80 to 443, and serves HTTP/2 and HTTP/3. There is no "enable TLS" line because there is no way to express "do not".

```
@exam path /me/tests/*/attempt /me/tests/*/brief /me/tests/*/paper \
        /me/attempts/*/paper /me/attempts/*/submit /me/attempts/*/state /me/leaderboard
```

A **named matcher**. It lists the exact leaf paths the exam role serves.

Why exact paths and not a prefix. The split between the two roles is not prefix-clean. `GET /me/performance` is exam, but `/me/performance/report`, `/me/performance/days` and `/me/performance/series` are core. `/me/attempts/*/question-report` is core, under the very same `/me/attempts` prefix that three exam routes use. A wildcard would therefore route live traffic to the wrong container — and it would do it silently, as a 404 rather than an error.

```
reverse_proxy @exam {
    dynamic a {
        name exam
        port 3000
        refresh 10s
    }
    fail_duration 30s
    max_fails 3
}
```

Requests matching `@exam` go to the `exam` service. `dynamic a` is **what makes scaling work**: it resolves the DNS name `exam` every 10 seconds, and Docker's internal resolver returns every container of that service. Run `--scale exam=4` and Caddy load-balances across four within ten seconds, with no edit to this file and no restart.

`fail_duration 30s` with `max_fails 3` is a **passive** health check: a container that fails three requests is taken out of rotation for thirty seconds and then tried again. Passive, not active — Caddy does not poll `/health` itself; Docker's own healthcheck does that.

```
reverse_proxy {
    dynamic a { name core ... }
}
```

No matcher, so this is the catch-all: everything that is not an exam path goes to `core`. Order matters — the first matching `reverse_proxy` wins, so the specific one must come first.

```
log {  
    output stdout  
    format json  
}
```

One JSON object per request, to stdout. Docker captures it, Alloy ships it to Loki, and the same 10 MB × 3 rotation that caps every other container caps this too. It used to be a file in a volume; that was before anything existed to carry it off the box.

13.2 Changing a route

Add a route to the exam role and you must add its path here, or it will go to `core`, which does not register it, and the student gets a 404 that looks like a missing feature.

```
vi deploy/Caddyfile  
sudo docker compose -f deploy/compose.yml --env-file deploy/.env exec caddy caddy validate --config  
/etc/caddy/Caddyfile  
sudo docker compose -f deploy/compose.yml --env-file deploy/.env restart caddy
```

Validate before restarting. A Caddyfile with a syntax error stops the container, and then nothing is serving.

13.3 What Caddy gives up against a load balancer

It is one program on one box. An AWS load balancer runs on several machines and heals itself; this does not. **A reboot of Box A is an outage.** At ten to thirty exam events a month that is something you schedule around rather than something you solve, and the saving is real — a load balancer is \$16–25 a month and it is the thing that would drag Fargate, target groups and a NAT back in with it.

The one escape hatch worth keeping open: a second box running the same compose file, with a Route 53 health check moving traffic to it. That is a \$0.50-a-month health check, not an architecture.

13.4 The certificate volume

`caddy-data` holds the issued certificates. **Do not delete it casually.** Let's Encrypt rate-limits five duplicate certificates per hostname per week, and a few careless `down -v` runs can lock you out of your own hostname for a week.

14. Grafana Cloud — logs and metrics

The `alloy` container already in `deploy/compose.yml` does both: it ships every container's stdout to Loki, and scrapes the API's `/metrics` into Prometheus. Nothing in the API knows it exists; it reads what Docker already captured.

14.1 Get the credentials

1. Sign up at **grafana.com** — the free tier is 50 GB of logs and 10,000 metric series, which is far more than this platform produces.
2. Your stack page → **Loki** → **Send Logs**. It shows a URL, a numeric username, and a button to generate a token. Create one with `logs:write`.
3. Same stack → **Prometheus** → **Send Metrics**. A URL, a **different** numeric username, and a token with `metrics:write`.

`PROM_USERNAME` is not `LOKI_USERNAME`. Grafana Cloud numbers each signal separately, and pasting one into the other fails with a 401 that reads exactly like a bad token. This costs people an hour.

14.2 Fill them in and restart

```
vi /opt/examprep/deploy/.env      # LOKI_URL, LOKI_USERNAME, LOKI_PASSWORD, PROM_*, ENVIRONMENT=staging
sudo docker compose -f deploy/compose.yml --env-file deploy/.env up -d alloy
sudo docker compose -f deploy/compose.yml --env-file deploy/.env logs --tail 30 alloy
```

14.3 What to query

In Grafana → **Explore** → **Loki**:

```
{env="staging"}                # everything
{env="staging", service_name="exam"}  # one role
{env="staging", level="error"}      # only failures
{service_name="caddy"} | json | status >= 500  # Caddy's view of 5xx
```

There are exactly three labels — `env`, `service_name` and `level` — and that is on purpose. A label is an index in Loki, and a label with many values makes queries slow and the bill large. Everything per-request stays inside the line and comes out at query time:

```
{service_name="exam", level="error"} |= "bug=" | pattern "<_>bug=<bug> <_>" | sum by (bug)
```

`service_name` is the compose service name — `exam`, `core`, `worker`, `caddy`, `migrate` — not the 64-character container id, which is the whole reason Alloy needs the Docker socket.

In **Explore** → **Prometheus**:

```
up{role="exam"}                # 1 per container; count(up) is your
replica count
rate(iace_http_requests_total{status=~"5.."}[5m])      # error rate
histogram_quantile(0.95, sum by (le) (rate(iace_http_request_duration_seconds_bucket[5m])))
```

`up == 0` is a container Alloy cannot scrape. `count(up{role="exam"})` after `--scale exam=4` should read 4 within fifteen seconds — nothing is listed anywhere, it is all discovered off the Docker API.

14.4 The alerts worth having on day one

Grafana → **Alerting** → **Alert rules**. Four, not forty:

Alert	Condition	Why
API down	<code>count(up{role="exam"}) == 0</code> for 2m	The thing that ends an exam
Error rate	5xx rate > 1% for 5m	Something broke under load
Queue backing up	oldest waiting job > 300s	Scoring is not draining; students see no results
Valkey memory	used / max > 0.8	Under <code>noeviction</code> , a full Valkey makes writes fail , which drops answers

Also set the two free **AWS Budgets** — one at your monthly total, one anomaly detector. Every alert above watches something that is running; none of them watches something that should have stopped, and on a first AWS account a forgotten test instance costs more than the mistakes the alerts catch.

15. Load testing

15.1 What exists

`scripts/load/exam-event.js` is a k6 script covering the five tiers of an exam event, chosen with `-e TIER=`:

Tier	What it simulates
sitting	The real thing: log in, start, autosave every 25s, submit
aftermath	Everyone opening scorecards, solutions and the leaderboard at once
login	The herd arriving in the ten minutes before a test opens
admin	An admin watching the live board while it happens
assets	Browsers pulling the site and question images

`scripts/bench-scoring.mjs` is separate and measures only the scoring drain, directly against the database.

Honest status: only `assets` and `login` have ever run, and only against a laptop. The sitting tier — the one that matters — has never been exercised, because it needs student accounts that can actually reach a startable test, and local development has one student and no such fixture. Getting that working on staging is the real task below.

15.2 Two things to change on the target first

Raise the auth rate limit. It is 120 requests a minute per IP address, and a load generator is a single IP. The herd hits that wall long before the box does, and your numbers measure the rate limiter.

```
# in deploy/.env, for the duration of the test only
RATE_LIMIT_AUTH_PER_MIN=100000
```

`RATE_LIMIT_SITTING_PER_MIN` needs no change: it is 60 per student and an autosave every 25 seconds is two or three.

Put it back afterwards. A staging box with the rate limiter off is a staging box somebody can brute-force.

15.3 Make the accounts

This is the part that has blocked the sitting tier. Staging runs `OTP_SENDER=console`, so the OTP comes back in the HTTP response — which means a script can create real students over the public API with no SMS provider at all:

```
#!/usr/bin/env bash
# make-load-accounts.sh 500 → writes accounts.csv
set -euo pipefail
API=https://api.staging.examprep.iace.co.in
COUNT=${1:-500}
BASE=9000000000
PIN=1234
echo "mobile,pin" > accounts.csv
for i in $(seq 0 $((COUNT - 1))); do
  MOBILE=$((BASE + i))
  CODE=$(curl -s -X POST "$API/auth/student/otp/request" \
    -H 'content-type: application/json' -H 'x-client: WEB' \
    -d '{"mobile":"$MOBILE"}' | jq -r '.data.devCode')
  TOKEN=$(curl -s -X POST "$API/auth/student/otp/verify" \
    -H 'content-type: application/json' -H 'x-client: WEB' \
    -d '{"mobile":"$MOBILE","code":"$CODE"}' | jq -r '.data.setupToken')
  curl -s -X POST "$API/auth/student/pin/set" \
    -H 'content-type: application/json' -H 'x-client: WEB' \
```

```
-d "{\"mobile\":\"\\\"$MOBILE\\\",\\\"setupToken\":\"\\\"$TOKEN\\\",\\\"pin\":\"\\\"$PIN\\\"}\" > /dev/null
echo \"$MOBILE,$PIN\" >> accounts.csv
done
```

Students created this way are `NON_IACE` with no branch, so **they reach nothing yet**. Give them something to sit, in the admin app:

1. Build one test with a real paper — a hundred questions is representative.
2. Put it in a **FREE** series, which every student reaches regardless of branch, or assign the whole set an explicit grant.
3. Set its window so it is open while you test.
4. Open it once as one of those students by hand. If one cannot start it, five thousand cannot.

15.4 Run it

Run k6 from a **second small EC2 instance in** `ap-south-1`, not from your laptop. From a laptop you measure your broadband and the path across the internet; from inside the region you measure the box, which is the thing you are trying to size. A `t3.small` for an hour is pennies.

```
k6 run -e BASE=https://api.staging.examprep.iace.co.in \
-e TIER=assets -e ASSET_RPS=20 -e DURATION=2m \
scripts/load/exam-event.js

k6 run -e BASE=https://api.staging.examprep.iace.co.in \
-e TIER=login -e LOGIN_PEAK_RPS=40 \
-e ACCOUNTS=./accounts.csv \
scripts/load/exam-event.js

k6 run -e BASE=https://api.staging.examprep.iace.co.in \
-e TIER=sitting -e CANDIDATES=500 \
-e OPEN_WINDOW_SEC=120 -e SITTING_SEC=600 \
-e ACCOUNTS=./accounts.csv -e TEST_ID=<the test id> \
scripts/load/exam-event.js
```

Climb: 100 candidates, then 500, then 1,000. The point of the small run is to find the broken fixture, not the capacity ceiling.

15.5 Reading the result

The script carries **thresholds**, so k6 passes or fails on its own terms rather than leaving you to interpret a wall of numbers. They are budgets taken from what the UI actually tolerates — the paper gives up on a save at 15 seconds and the API refuses at 8:

Measure	Budget
<code>http_req_failed</code>	under 1%
<code>autosave_applied</code>	over 99%
autosave p95	500 ms
submit p95	2 s
test paper p95	3 s
student login p95	1 s

`throttled_429` **is counted separately on purpose**. A rate-limit rejection is not the API failing, and without that separation a run that hit the limiter reads like a run that fell over. If that counter is non-zero, §15.2 was skipped.

While it runs, watch the box:

```
sudo docker stats          # per-container CPU and memory
```

and watch Grafana for the queue depth and the oldest waiting job.

15.6 What the numbers decide

One number settles the production instance size. Simulate the event at the intended production size and read the CPU at peak:

Peak CPU	Verdict
under 50%	The size is right, with room
50–80%	Go one size up for events
over 80%	Two sizes up, and reconsider the container split

And then **write what you measured into** `docs/04-infrastructure.md` §16, which currently lists every figure that was never measured. The largest of them is the assumption that a Graviton core is three to four times slower than the laptop these benchmarks were taken on. Every projection in that file rests on it, and one run of `scripts/bench-scoring.mjs` on this box replaces it with a fact.

16. Scaling

Four separate dials. Reach for them in this order.

16.1 More containers — seconds, free

```
sudo docker compose -f deploy/compose.yml --env-file deploy/.env up -d --scale exam=4
```

Caddy picks them up within ten seconds through `dynamic a`, and Alloy starts scraping them off the same Docker API. **Nothing is configured anywhere.**

The worker is excluded, always. `--scale worker=2` puts two sweeps over the same Valkey keys with nothing coordinating them. If scoring ever needs to scale, the answer is a fourth role that registers only the scoring processor — not a second worker.

Watch the database when you do this. Each `exam` container opens 8 connections and each `core`

1. A `db.t4g.micro` has about 112. Scaling `exam` to 4 is the thing that exhausts it, and the symptom is a connection error under load rather than slowness.

16.2 A bigger box — two minutes, hourly billing

```
sudo docker compose -f deploy/compose.yml --env-file deploy/.env down
# Console → EC2 → instance → Actions → Instance state → Stop
# → Actions → Instance settings → Change instance type → t4g.xlarge → Start
sudo docker compose -f deploy/compose.yml --env-file deploy/.env up -d
```

The Elastic IP survives. EBS survives. Billing is hourly, so running four times the box for the four hours of an event costs about a third of a dollar.

Size	vCPU	RAM	When
<code>t4g.small</code>	2	2	Staging once images come from ECR
<code>t4g.medium</code>	2	4	Staging today — you build on the box
<code>t4g.large</code>	2	8	Production baseline
<code>t4g.xlarge</code>	4	16	A scheduled event
<code>t4g.2xlarge</code>	8	32	Roughly eight times the 10K ceiling

Pre-warming is deliberate, not automatic. There is no autoscaling group here. Resizing before a window goes on the release calendar — and it should, because the alternative is discovering the need while 8,000 people are typing.

`t4g` **is burstable, and unlimited mode is the default.** Past a 20%-per-vCPU baseline you are billed a surcharge rather than throttled — about \$0.04 per surplus vCPU-hour. That is the right trade for an exam, because a bill beats an outage, but it is the single largest gap between the expected invoice and the worst case. Put a CloudWatch alarm on `CPUSurplusCreditsCharged`.

16.3 Valkey and the database

Piece	Now	Moves to	Trigger
Box B	<code>t4g.micro</code>	<code>t4g.small</code>	Before the first full-scale load test
		<code>t4g.medium</code> , 20 GB	Production carrying 8,000 live sittings
RDS	<code>db.t4g.micro</code>	<code>db.t4g.small</code>	A second <code>exam</code> container — connections, not load

What sizes Box B is not the dataset. Valkey is single-threaded and an event is about 960 operations a second against a core that does 100,000, so CPU never decides. Memory does — and specifically the **AOF rewrite fork**: Valkey rewrites its append-only log by forking, and copy-on-write can push resident memory towards double the dataset while writes are still landing. Size for twice what you hold.

16.4 Deploying without a gap

`up -d` recreates containers with a gap. To avoid it:

```
sudo docker compose ... up -d --scale exam=2 --no-recreate  
sudo docker compose ... up -d --force-recreate --no-deps exam
```

That is the cost of having no load balancer, stated plainly. For a staging box it does not matter.

17. PostgreSQL — backups and restore

17.1 What you already have

Point-in-time restore, 7 days. Turned on in §5. RDS continuously ships write-ahead logs, so you can restore to any second in that window. It is free up to your allocated storage, and it is the one that saves you from a bad migration or a wrong `DELETE`.

A restore does **not** overwrite the instance. It creates a *new* instance from the chosen moment, which you then point the application at. That is slower to think about and much safer.

```
aws rds restore-db-instance-to-point-in-time \  
  --source-db-instance-identifier examprep-db \  
  --target-db-instance-identifier examprep-db-restored \  
  --restore-time 2026-10-02T09:30:00Z \  
  --db-subnet-group-name <your-subnet-group> \  
  --vpc-security-group-ids sg-xxxxxxx \  
  --no-publicly-accessible
```

Then change `DATABASE_URL` on Box A to the new endpoint and restart.

17.2 A manual snapshot before anything risky

Before a migration that moves data, or any release you are unsure of:

```
aws rds create-db-snapshot \  
  --db-instance-identifier examprep-db \  
  --db-snapshot-identifier examprep-db-pre-release-$(date +%Y%m%d-%H%M)
```

Manual snapshots are kept until you delete them, unlike the automatic ones.

17.3 A logical dump, which the other two do not replace

A snapshot and a PITR both restore a whole instance. Neither lets you pull one table back, read the data elsewhere, or survive the AWS account itself being lost. A weekly `pg_dump` to S3 covers all three:

```
# on Box A, from cron  
pg_dump "$DATABASE_URL" --format=custom --no-owner --no-acl \  
  --file=/tmp/examprep_staging_$(date +%F).dump  
aws s3 cp /tmp/examprep_staging_$(date +%F).dump \  
  s3://examprep-staging-media/backups/ --storage-class STANDARD_IA  
rm /tmp/examprep_staging_*.dump
```

Restoring one table from it:

```
pg_restore --dbname "$DATABASE_URL" --table=Student --data-only examprep_staging_2026-10-02.dump
```

17.4 Rehearse it once, on staging, this month

A backup nobody has restored is not a backup. Restore staging from a point-in-time to a new instance, point the box at it, confirm the admin can sign in, then delete it. An hour, and it turns the procedure above from a document into something you have done.

That is also exactly why staging runs with 7-day PITR even though nobody would miss staging's data: so the rehearsal is possible without touching production.

17.5 Never restore production into staging

Real students' names, mobile numbers and marks do not belong in a lower-security environment — and while the two share one RDS instance and one Valkey box, that rule is doing more work than usual. Staging gets seeded and generated data only.

18. Valkey — connecting and looking inside

18.1 Connect

From Box B:

```
sudo docker compose -f deploy/valkey/compose.yml exec valkey-staging valkey-cli
```

`REDISCLI_AUTH` is already in that container's environment, so the shell is authenticated and the password never appears in a command line or in `~/.bash_history`. From Box A, over the network:

```
sudo docker run --rm -it -e REDISCLI_AUTH=<password> valkey/valkey:8-alpine \
valkey-cli -h 10.0.1.20 -p 6380
```

18.2 What to look at

```
INFO memory           # used_memory_human against maxmemory — the number that matters
INFO persistence      # aof_enabled 1, aof_last_write_status ok
INFO clients           # connected_clients: your containers
DBSIZE                # how many keys
SCAN 0 MATCH bull:* COUNT 20
SCAN 0 MATCH sitting:* COUNT 20
```

Never run `KEYS *` on a box with live sittings. It is single-threaded and `KEYS` walks the whole keyspace, blocking everything else for as long as it takes. `SCAN` is the one that is safe under load.

18.3 The settings, and why each is not negotiable

Setting	Value	Why
<code>maxmemory-policy</code>	<code>noeviction</code>	BullMQ requires it, and this holds live answers. Any eviction policy silently drops a student's work. The API refuses to boot in production against anything else
<code>maxmemory</code>	always set explicitly	Unset, Valkey grows until the kernel kills it, and an OOM-killed Valkey loses up to a second of answers
<code>appendonly</code>	<code>yes</code> , every second	A restart loses at most that second instead of everything
<code>requirepass</code>	set, one per process	§6.3

One process per environment, not one process with two database indexes. An index would separate the keyspaces — which matters, because BullMQ has no prefix configured and both environments would otherwise share `bull:scoring:*`, letting a production worker pick up a staging job. But `maxmemory` is per *process*, and under `noeviction` an instance that fills makes writes **fail**. A separate process means a staging runaway cannot stop production saving an answer.

18.4 When it dies

Losing Valkey loses **in-flight sittings**, not scored ones. Answers already flushed to Postgres and every scored attempt are safe. Students mid-paper lose up to their last autosave.

```
sudo docker compose -f deploy/valkey/compose.yml logs --tail 100 valkey-staging
sudo docker compose -f deploy/valkey/compose.yml up -d valkey-staging
```

The AOF replays on start, so it comes back with what it had.

18.5 Why ElastiCache is not used

Serverless has no parameter groups, so `maxmemory-policy` cannot be set at all — which BullMQ's own documentation rules out. A node-based `cache.t4g.small` is \$23.94 against this box's \$4.09, and it comes back **empty** after a node failure, because no current ElastiCache engine writes to disk.

19. Production — what is different

Production is this document again, with these changes. **Build it only once staging has run a real load test**, because the one number you do not have yet is what size Box A should be.

	Staging	Production
Box A	t4g.medium → small	t4g.large
Box B	shared, port 6380	same box, port 6379 , second process
RDS	examprep_staging on examprep-db	examprep_production, same instance, db.t4g.small
NODE_ENV	development	production
OTP_SENDER	console	sms
Heaps	384 MB each	exam 1536, core 768, worker 768
Memory	1g each	exam 2g, core 1g, worker 1g
Hostname	api.staging.examprep.iace.co.in	api.examprep.iace.co.in
SSM parameter	/examprep/staging/env	/examprep/production/env
Secrets	generated	different generated values

19.1 What production refuses to start without

Switching `NODE_ENV` turns on a set of boot-time refusals. Each names its own variable rather than failing at the first login:

- `DATABASE_URL` without `connection_limit` — the default pool is smaller than the 20 worker slots one container runs
- an empty `CORS_ORIGINS` — an empty allowlist lets any site call the API
- a missing `METRICS_TOKEN` — `/metrics` would answer anybody who asked, with live attempt counts
- `TRUST_PROXY_HOPS` at 0 — every caller would share one rate-limit bucket and the hall would lock itself out
- a Valkey that can evict
- any of `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET` or `PIN_PEPPER` still carrying the `dev_only_` prefix that `.env.example` publishes
- `OTP_SENDER=console`, and `sms` without `SMS_PROVIDER_URL`, `MAIL_USER` and `MAIL_PASSWORD`

It also **warns**, without refusing, when a container's heap does not match its memory limit, naming the role.

19.2 Release order

1. Build both images from `apps/api/Dockerfile` — `runtime` and `migrate`.
2. Build the sites. `VITE_API_URL` is compile-time, so this is a production-only artifact.
3. Upload hashed assets **first**, then `index.html`, `sw.js` and the manifest with `no-cache`.
4. Invalidate those three paths only — never `/*`.
5. Run the `migrate` image to completion.
6. Recreate the API containers.
7. If `deploy/.env` changed, push it back to SSM in the same breath.

A new service worker installs but does not activate until every tab of the old one closes. There is no `skipWaiting`, deliberately: a bundle swapped under a sitting in progress is worse than a stale tab. Deploying and students seeing the new build are different moments.

Never during an event window, and never a migration that moves data without rehearsing it against a copy at the previous revision first.

20. The SMS vendor

Until this is done, students cannot sign in on production. It is the longest-lead item in this document — DLT registration takes **one to two weeks** — so start it today even though nothing else here depends on it.

20.1 What DLT is

India's telecom regulator requires every commercial SMS sender to register on a blockchain-backed registry called DLT. You register three things: your **entity** (the institute, with its GST certificate), your **header** (a six-character sender id such as `IACEDU`), and each **template** you will ever send, with variables marked. A message that does not exactly match an approved template is rejected by the carrier, not by your code.

20.2 The steps

1. Pick an aggregator — MSG91, Gupshup, Kaleyra, Textlocal are the common Indian ones. Ask for **transactional** routing; promotional is blocked on DND numbers and OTPs are not promotional.
2. Register the entity on the aggregator's DLT portal with the institute's GST and PAN.
3. Register the header `IACEDU` (or similar). **Approval takes days.**
4. Register one template per message kind. Keep them short; every variable is a `{#var#}`:

Kind	Template	Env var
OTP	<code>{#var#}</code> is your IACE verification code. Valid 5 minutes. Do not share.	<code>SMS_TEMPLATE_OTP</code>
PIN	Your IACE PIN has been reset. Set a new one at <code>{#var#}</code>	<code>SMS_TEMPLATE_PIN</code>
Result ready	Your result for <code>{#var#}</code> is ready. IACE	<code>SMS_TEMPLATE_RESULT_READY</code>
Test assigned	A new test <code>{#var#}</code> is available. IACE	<code>SMS_TEMPLATE_TEST_ASSIGNED</code>
Reminder	<code>{#var#}</code> opens at <code>{#var#}</code> . IACE	<code>SMS_TEMPLATE_TEST_REMINDER</code>
Announcement	<code>{#var#}</code>	<code>SMS_TEMPLATE_ANNOUNCEMENT</code>

1. The aggregator gives each approved template a **template id**. Those ids go in the env vars, not the text.

20.3 Wiring it

```
OTP_SENDER=sms
SMS_PROVIDER_URL=https://api.msg91.com/api/v5/flow/
SMS_PROVIDER_KEY=<auth key>
SMS_SENDER_ID=IACEDU
SMS_TEMPLATE_OTP=<approved id>
SMS_TEMPLATE_PIN=<approved id>
# the rest are optional; a kind with no template is OFF, not broken
```

The provider is deliberately a **shape, not a name**: endpoint, key, sender id and a template id per kind. Every Indian aggregator works this way, so switching vendor is a config change rather than a code change.

OTP and PIN are required; the other four are optional. A kind with no template configured is treated as switched off, and the caller is told so rather than the API recording a message as delivered that was never sent.

The budget is enforced in code. `OTP_GLOBAL_DAILY_BUDGET_PAISE` defaults to ₹3,000 a day, and `NOTIFICATION_COST_SMS_PAISE` is 18 paise a message. That is a stop on a loop or an attack costing a fortune overnight.

21. Email

An **admin** signs in with an emailed OTP, so email is required the moment `OTP_SENDER` is not `console` — even if SMS is not ready. It is Gmail through nodemailer's well-known service configuration, so there is nothing to host.

1. The sending account must have **2-Step Verification** on.
2. Google Account → Security → **App passwords** → generate one for "Mail". It is 16 characters.
3. Put it in the env file:

```
MAIL_USER=noreply@iace.co.in
MAIL_PASSWORD=<the 16-character app password>
```

It is not the account password. A normal password will not work; Google blocks it, and the error is an unhelpful authentication failure.

For volume beyond a few hundred a day, move to Amazon SES in `ap-south-1` — but SES starts in a sandbox that only sends to verified addresses, and leaving it needs a support request that takes a day or two. Gmail is correct for now.

The API refuses to boot with `OTP_SENDER=sms` and no `MAIL_USER`, deliberately: discovering that admins cannot sign in at the first admin login is the worst possible moment.

22. When something is wrong

Symptom	Almost always	Check
Caddy gets no certificate	DNS or port 80	<code>dig +short api.staging.example.iace.co.in</code> ; port 80 open to <code>0.0.0.0/0</code> ; <code>API_HOST</code> matches exactly
API container restarts in a loop	A refused env var	<code>logs exam</code> — the refusal names the variable
<code>unhealthy</code> containers	A dependency is unreachable	<code>curl localhost:3000/health/ready</code> inside the container; then the security groups
Valkey <code>unhealthy</code>	Password mismatch	<code>REDISCLI_AUTH</code> must equal <code>--requirepass</code> ; both come from <code>deploy/valkey/.env</code>
API cannot reach Valkey	The URL form	<code>redis://:PASSWORD@host:6380</code> — the empty username before the colon is required
Database connection refused	Security group	RDS must admit 5432 from <code>example-api</code> , by group not by address
Site loads, every call fails	<code>CORS_ORIGINS</code>	Exact origins, scheme included, no trailing slash. The error is only in the browser console
Site 404s on a deep link	The CloudFront Function	§11.4
Exit 143	A clean shutdown	Normal
Exit 137	SIGKILL	<code>docker inspect</code> → <code>OOMKilled</code> true means memory; false means the grace period expired
Everything rate-limited	<code>TRUST_PROXY_HOPS</code> is 0	At 0 every caller shares one bucket, because they all arrive from Caddy
Alloy ships nothing	The two usernames	<code>PROM_USERNAME</code> is not <code>LOKI_USERNAME</code>

Reading logs:

```
cd /opt/example
C="sudo docker compose -f deploy/compose.yml --env-file deploy/.env"
$C ps
$C logs --tail 100 exam
$C logs -f caddy
$C exec exam sh          # a shell inside a container
$C restart exam
$C down && $C up -d --build # the big hammer
sudo docker stats        # live CPU and memory
df -h                    # the disk, which fills quietly
```

23. After today

This week	Run the full load test (§15); replace <code>docs/04</code> §16's unmeasured figures with real ones
This week	Rehearse a point-in-time restore (§17.4)
This week	Start the DLT registration (§20) — it is the long pole
Next	Custom domains on the two sites (§12.6)
Next	Build images in CI, push to ECR, and drop Box A' to <code>t4g.small</code>
Next	The four Grafana alerts and the two AWS Budgets (§14.4)
Before production	Decide Box A's size from the load test, not from this document

One habit worth forming now. Every number in `docs/04-infrastructure.md` carries the date it was measured and the command that measured it, and §16 lists the ones that are still assumptions. When you measure something today, write it there. The largest assumption in the whole file is that a Graviton core is three to four times slower than the laptop these benchmarks came from — every cost and capacity projection rests on it, and one run of `scripts/bench-scoring.mjs` on Box A replaces it with a fact.